

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN



BÀI TẬP LỚN

MÔN: HỌC MÁY CƠ BẢN
DỰ ĐOÁN NGUY CƠ MẮC BỆNH TIỂU ĐƯỜNG BẰNG
LOGISTIC REGRESSION, RANDOM FOREST

NGÀNH: KHOA HỌC MÁY TÍNH
CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

SINH VIÊN: Phạm Tùng Dương
MÃ SINH VIÊN: 10123071
MÃ LỚP: 12423TN
GIẢNG VIÊN HƯỚNG DẪN: PGS.TS NGUYỄN VĂN HẬU

HƯNG YÊN – 2025

[illegible]

LỜI CAM ĐOAN

Em xin cam đoan bài tập lớn môn Học máy cơ bản là kết quả thực hiện của chúng em dưới sự hướng dẫn của thầy Nguyễn Văn Hậu

Những phần sử dụng tài liệu tham khảo trong bài tập lớn đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong bài tập lớn và chương trình xây dựng được hoàn toàn là kết quả do bản thân em thực hiện.

Nếu vi phạm lời cam đoan này, em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng yên, ngày 28 tháng 12 năm 2025

Sinh viên

Phạm Tùng Dương

LỜI CẢM ƠN

Để có thể hoàn thành bài tập lớn này, lời đầu tiên em xin phép gửi lời cảm ơn tới bộ môn Khoa học máy tính, Khoa Công nghệ thông tin – Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho em thực hiện bài tập lớn môn học này.

Đặc biệt em xin chân thành cảm ơn thầy Nguyễn Văn Hậu đã rất tận tình hướng dẫn, chỉ bảo em trong suốt thời gian thực hiện bài tập lớn vừa qua.

Mặc dù chúng em đã có cố gắng, nhưng với trình độ còn hạn chế, trong quá trình thực hiện không tránh khỏi những thiếu sót. Em hy vọng sẽ nhận được những ý kiến nhận xét, góp ý của cô về những kết quả triển khai trong bài tập lớn.

Em xin trân trọng cảm ơn!

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN.....	9
1.1 Bài toán.....	9
1.2 Trình bày dữ liệu bài toán.....	9
1.3 Tiền xử lý dữ liệu.....	11
1.4 Trực quan dữ liệu.....	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	13
2.1 Pandas.....	13
2.1.1 Giới thiệu về Pandas.....	13
2.1.2 Cài đặt Pandas	13
2.2 Numpy.....	14
2.2.1 Giới thiệu về Numpy.....	14
2.2.2 Cài đặt Numpy.....	14
2.3 Matplotlib	14
2.3.1 Giới thiệu về Matplotlib.....	14
2.3.2 Cài đặt Matplotlib.....	15
2.4 Random Forset.....	15
2.4.1 Cấu trúc của Random Forest.....	15
2.4.2 Cách thức hoạt động của Random Forest.....	16
2.4.3 Ứng dụng của Random Forest.....	16
2.5 Logistic Regression.....	16
2.5.1 Cấu trúc của Logistic Regression.....	17
2.5.2 Cách thức hoạt động của Logistic Regression.....	17

2.5.3 Ứng dụng của Logistic Regression.....	17
2.5.4 Loss Function.....	18
2.5.5 Các loại Logistic Regression.....	18
CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ.....	19
3.1 Mã nguồn tiền xử lý dữ liệu.....	20
3.2 Mã nguồn thống kê dữ liệu.....	22
3.3 Mã nguồn trực quan hóa dữ liệu.....	25
3.4 Áp dụng model.....	41
3.4.1 Model Logistic Regression.....	44
3.4.2 Model Random Forest.....	46
3.4.3 So sánh và đánh giá mô hình.....	47
3.5 Sản phẩm.....	48
KẾT LUẬN.....	51
TÀI LIỆU THAM KHẢO.....	52

DANH MỤC HÌNH ẢNH

HÌNH 1.1: DỮ LIỆU BỘ DATASET DIABD: A DATASET ON DIABETES IN BANGLADESH.....	9
HÌNH 1.2: THÔNG TIN TỪNG CỘT CỦA DATASET.....	9
HÌNH 1.3: MÔ TẢ DỮ LIỆU TỪNG CỘT CỦA DATASET.....	10
Hình 3.1: THÔNG TIN TỪNG CỘT DATASET.....	20
HÌNH 3.2: LỌC VÀ MÃ HÓA NHÃN.....	21
HÌNH 3.3: MÃ NGUỒN XỬ LÝ GIÁ TRỊ NGOẠI LẠI.....	21
HÌNH 3.4: KIỂM TRA & XỬ LÝ GIÁ TRỊ THIỂU.....	22
HÌNH 3.5: MÃ NGUỒN ĐẾM SỐ LƯỢNG GIỚI TÍNH.....	22
HÌNH 3.6: MÃ NGUỒN ĐẾM SỐ LƯỢNG MẮC BỆNH.....	23
HÌNH 3.7: MÃ NGUỒN ĐẾM SỐ LƯỢNG NGƯỜI CÓ MỨC GLUCOSE CAO.....	23
HÌNH 3.8: MÃ NGUỒN ĐẾM SỐ LƯỢNG NGƯỜI CÓ MỨC BMI CAO.....	24
HÌNH 3.9: MÃ NGUỒN BIỂU ĐỒ HISTPLOT PHÂN BỐ MỨC GLUCOSE.....	25
HÌNH 3.10: BIỂU ĐỒ HISTPLOT HIỂN THỊ PHÂN BỐ GLUCOSE.....	26
HÌNH 3.11: MÃ NGUỒN BIỂU ĐỒ HEATMAP.....	26
HÌNH 3.12: BIỂU ĐỒ TƯƠNG QUAN DỮ LIỆU.....	27
HÌNH 3.13: MÃ NGUỒN VẼ BIỂU ĐỒ MẮC BỆNH THEO NHÓM TUỔI.....	28
HÌNH 3.14: BIỂU ĐỒ BARS HIỂN THỊ MỨC ĐỘ MẮC BỆNH THEO NHÓM TUỔI.....	29
HÌNH 3.15: MÃ NGUỒN BIỂU ĐỒ BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO GIỚI TÍNH.....	29
HÌNH 3.16: BIỂU ĐỒ BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO GIỚI TÍNH.....	30
HÌNH 3.17: MÃ NGUỒN BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO GIA ĐÌNH.....	31
HÌNH 3.18: BIỂU ĐỒ BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO GIA ĐÌNH.....	31
HÌNH 3.19: MÃ NGUỒN BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO ĐỘT QUY.....	32
HÌNH 3.20: BIỂU ĐỒ BARPLOT HIỂN THỊ TỈ LỆ MẮC BỆNH THEO ĐỘT QUY.....	33
HÌNH 3.21: MÃ NGUỒN BIỂU ĐỒ NGUY CƠ TIỂU ĐƯỜNG THEO ĐƯỜNG HUYẾT.....	33
HÌNH 3.22: BIỂU ĐỒ NGUY CƠ TIỂU ĐƯỜNG THEO ĐƯỜNG HUYẾT.....	34
HÌNH 3.23: MÃ NGUỒN BIỂU ĐỒ NGUY CƠ TIỂU ĐƯỜNG THEO BMI.....	35
HÌNH 3.24: BIỂU ĐỒ NGUY CƠ TIỂU ĐƯỜNG THEO BMI.....	36
HÌNH 3.25: MÃ NGUỒN BIỂU ĐỒ CÁC YẾU TỐ QUAN TRỌNG.....	36
HÌNH 3.26: BIỂU ĐỒ CÁC FEATURE QUAN TRỌNG.....	37
HÌNH 3.27: MÃ NGUỒN BIỂU ĐỒ MẮC BỆNH TIỂU ĐƯỜNG.....	38
HÌNH 3.28: BIỂU ĐỒ MẮC BỆNH TIỂU ĐƯỜNG TRONG DỮ LIỆU.....	39
HÌNH 3.29: CHIA DATASET RA LÀM HAI PHẦN ĐỂ TRAIN VÀ TEST.....	39
HÌNH 3.30: KẾT QUẢ KHI CHIA DATASET RA LÀM HAI TẬP.....	40
HÌNH 3.31: CHUẨN HÓA DỮ LIỆU.....	40

HÌNH 3.32: SMOTE DỮ LIỆU	41
HÌNH 3.33: KẾT QUẢ SAU KHI SMOTE DỮ LIỆU	41
HÌNH 3.34: MÃ NGUỒN HUẤN LUYỆN MÔ HÌNH.....	43
HÌNH 3.35: MÃ NGUỒN MODEL LOGISTIC REGRESSION TRƯỚC SMOTE.....	44
HÌNH 3.36: KẾT QUẢ MODEL LOGISTIC REGRESSION TRƯỚC SMOTE.....	44
HÌNH 3.37: MÃ NGUỒN MODEL LOGISTIC REGRESSION SAU SMOTE.....	45
HÌNH 3.38: KẾT QUẢ MODEL LOGISTIC REGRESSION SAU SMOTE.....	45
HÌNH 3.39: MÃ NGUỒN MODEL RANDOM FOREST	46
HÌNH 3.40: SO SÁNH HAI MODEL.....	46
HÌNH 3.41: TIÊU ĐỀ SẢN PHẨM.....	48
HÌNH 3.42: GIAO DIỆN SẢN PHẨM.....	49
HÌNH 3.43: KẾT QUẢ CHẠY CỦA MỘT VÍ DỤ DỰ ĐOÁN BẰNG LOGISTIC REGRESSION	50

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN

1.1 Bài toán

Lĩnh vực y tế kỹ thuật số, đặc biệt là việc ứng dụng học máy trong dự đoán bệnh lý mãn tính, đang trở thành một trong những giải pháp quan trọng nhằm nâng cao chất lượng sống và giảm tải cho hệ thống y tế. Bằng cách phân tích dữ liệu sức khỏe và các chỉ số sinh học, chúng ta có thể thu thập thông tin chi tiết về các yếu tố nguy cơ, xu hướng phát triển bệnh và khả năng mắc bệnh tiểu đường của từng cá nhân. Dữ liệu này thường được tổng hợp từ các hồ sơ bệnh án, các cuộc khảo sát sức khỏe cộng đồng và các chỉ số xét nghiệm lâm sàng uy tín.

Việc phân tích dữ liệu này không chỉ giúp người dân chủ động hơn trong việc tầm soát sức khỏe mà còn cung cấp cái nhìn tổng quan về tác động của các yếu tố như chỉ số khối cơ thể (BMI), mức đường huyết (Glucose), huyết áp và thói quen sinh hoạt đến nguy cơ mắc bệnh. Các mô hình dự đoán được cập nhật liên tục dựa trên dữ liệu thực tế, đảm bảo tính chính xác và độ tin cậy, giúp phân loại mức độ rủi ro từ bình thường đến cảnh báo đỏ.

Ngoài ra, việc dự báo sớm nguy cơ bệnh tật có ảnh hưởng lớn đến chiến lược y tế dự phòng của cả nước. Việc dự đoán chính xác không chỉ giúp người bệnh có kế hoạch điều chỉnh lối sống kịp thời mà còn hỗ trợ các chuyên gia y tế trong việc tối ưu hóa phác đồ điều trị và phân bổ nguồn lực. Thông qua các công cụ phân tích dữ liệu, người dùng có thể đưa ra quyết định chăm sóc sức khỏe phù hợp, từ đó giảm thiểu biến chứng và tìm được hướng điều trị tối ưu nhất cho bản thân.

1.2 Trình bày dữ liệu bài toán

Dữ liệu bài toán được chọn lọc và lấy từ website có đường địa chỉ như sau: <https://data.mendeley.com/datasets/m8cgvxs9s6/2> có dạng csv với tên gọi: DiaBD A diabetes dataset aimed at enhancing risk analysis and research in Bangladesh.

age	gender	pulse_rate	systolic_bp	diastolic_bp	glucose	height	weight	bmi	family_diabetes	hypertensive	family_hypertension	cardiovascular_disease	stroke
42	Female	66	110	73	5.88	1.65	70.2	25.75	0	0	0	0	0
35	Female	60	125	68	5.71	1.47	42.5	19.58	0	0	0	0	0
62	Female	57	127	74	6.85	1.52	47.0	20.24	0	0	0	0	0
73	Male	55	193	112	6.28	1.63	57.4	21.72	0	0	0	0	0
68	Female	71	150	81	5.71	1.42	36.0	17.79	0	0	0	0	0
40	Male	48	131	73	7.22	1.63	61.9	23.42	0	0	0	0	0
35	Female	66	123	72	5.40	1.42	42.3	20.91	0	0	0	0	0
25	Female	84	152	90	7.20	1.52	57.7	24.84	0	0	0	0	0
50	Male	80	135	87	17.04	1.68	67.8	24.13	0	0	0	0	0
70	Male	64	129	75	8.04	1.65	56.0	20.54	0	0	0	0	0

Hình 1.1: Dữ liệu bộ Dataset DiaBD: A dataset on diabetes in Bangladesh

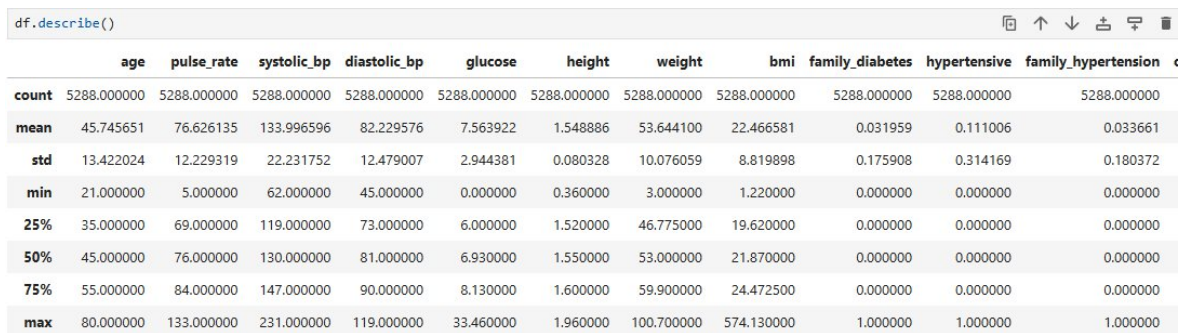
```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5288 entries, 0 to 5287
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    5288 non-null   int64
1   gender                 5288 non-null   object
2   pulse_rate             5288 non-null   int64
3   systolic_bp            5288 non-null   int64
4   diastolic_bp           5288 non-null   int64
5   glucose                5288 non-null   float64
6   height                 5288 non-null   float64
7   weight                 5288 non-null   float64
8   bmi                   5288 non-null   float64
9   family_diabetes        5288 non-null   int64
10  hypertensive            5288 non-null   int64
11  family_hypertension     5288 non-null   int64
12  cardiovascular_disease  5288 non-null   int64
13  stroke                 5288 non-null   int64
14  diabetic               5288 non-null   object
dtypes: float64(4), int64(9), object(2)
```

Hình 1.2: Thông tin từng cột của Dataset

- Dữ liệu bài toán gồm các cột:
 - Age: tuổi
 - Gender: giới tính
 - Pulse_rate: nhịp tim
 - Systolic_bp: huyết áp tâm thu
 - Diastolic_bp: huyết áp tâm trương

- Glucose: mức đường huyết
 - Height: chiều cao
 - Weight: cân nặng
 - Bmi: chỉ số khối cơ thể
 - Family_diabetes: tiền sử bị bệnh tiểu đường trong gia đình
 - Hypertensive: cá nhân mắc bệnh tăng huyết áp
 - Family_hypertension: tiền sử mắc bệnh tăng huyết áp trong gia đình
 - Cardiovascular_disease: cá nhân mắc bệnh tim mạch
 - Stroke: cá nhân mắc bệnh đột quỵ
- Dữ liệu bài toán là một file csv gồm: 5287 rows x 14 columns
 - File csv: DiaBD A dataset on diabetes in Bangladesh
 - Có 14 cột và 5287 bản ghi
 - Dữ liệu sau khi mô tả, ta có :



	age	pulse_rate	systolic_bp	diastolic_bp	glucose	height	weight	bmi	family_diabetes	hypertensive	family_hypertension	
count	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	5288.000000	
mean	45.745651	76.626135	133.996596	82.229576	7.563922	1.548886	53.644100	22.466581	0.031959	0.111006	0.033661	
std	13.422024	12.229319	22.231752	12.479007	2.944381	0.080328	10.076059	8.819898	0.175908	0.314169	0.180372	
min	21.000000	5.000000	62.000000	45.000000	0.000000	0.360000	3.000000	1.220000	0.000000	0.000000	0.000000	
25%	35.000000	69.000000	119.000000	73.000000	6.000000	1.520000	46.775000	19.620000	0.000000	0.000000	0.000000	
50%	45.000000	76.000000	130.000000	81.000000	6.930000	1.550000	53.000000	21.870000	0.000000	0.000000	0.000000	
75%	55.000000	84.000000	147.000000	90.000000	8.130000	1.600000	59.900000	24.472500	0.000000	0.000000	0.000000	
max	80.000000	133.000000	231.000000	119.000000	33.460000	1.960000	100.700000	574.130000	1.000000	1.000000	1.000000	

Hình 1.3: Mô tả dữ liệu từng cột của Dataset

- Trong Dataset cần chuẩn hóa dữ liệu đầu vào và xử lý bất thường dữ liệu.

1.3 Tiền xử lý dữ liệu

1. Xử lý thông tin các cột, gender (giới tính), diabetic (mắc bệnh)
2. Loại bỏ dữ liệu ngoại lai
3. Chuẩn hóa dữ liệu cột :age
4. Chuẩn hóa dữ liệu cột:pulse_rate
5. Chuẩn hóa dữ liệu cột: diastolic_bp
6. Chuẩn hóa dữ liệu cột: systolic_bp

7. Chuẩn hóa dữ liệu cột: glucose
8. Chuẩn hóa dữ liệu cột: bmi

1.4 Trực quan dữ liệu

1. Biểu đồ figure phân bố ngưỡng quyết định mắc bệnh tiểu đường
2. Biểu đồ tương quan
3. Biểu đồ phân bố mắc bệnh tiểu đường theo nhóm tuổi
4. Biểu đồ phân bố mắc bệnh tiểu đường theo giới tính
5. Biểu đồ phân bố mắc bệnh tiểu đường theo tiền sử gia đình
6. Biểu đồ phân bố mắc bệnh tiểu đường theo tiền sử đột quỵ
7. Biểu đồ phân bố mắc bệnh theo mức đường huyết
8. Biểu đồ phân bố mắc bệnh theo chỉ số BMI
9. Biểu đồ tỉ lệ mắc bệnh trong dữ liệu

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Pandas

2.1.1 Giới thiệu về Pandas

Pandas là một thư viện mã nguồn mở phổ biến trong Python, được xây dựng trên nền tảng NumPy, chuyên dùng để thao tác và phân tích dữ liệu. Với khả năng xử lý các loại dữ liệu đa dạng như chuỗi thời gian, bảng không đồng nhất và ma trận dữ liệu, Pandas mang lại trải nghiệm làm việc trực quan và linh hoạt cho người dùng.

Thư viện này cho phép import dữ liệu từ nhiều nguồn như CSV, cơ sở dữ liệu SQL, và các định dạng đặc biệt khác. Sau khi nhập dữ liệu, người dùng có thể thực hiện các thao tác như chọn lọc (subsetting), cắt nhỏ (slicing), lọc (filtering), hợp nhất (merging), nhóm (groupBy), sắp xếp (re-ordering), và tái cấu trúc (re-shaping) dữ liệu một cách dễ dàng. Pandas cũng hỗ trợ xử lý dữ liệu mất mát bằng cách loại bỏ hoặc thay thế bằng giá trị khác.

Ngoài ra, Pandas tích hợp tốt với các thư viện như Matplotlib, SciPy và Scikit-Learn, giúp mở rộng khả năng phân tích và trực quan hóa. Với hiệu suất cao, Pandas có thể xử lý khối lượng dữ liệu lớn hiệu quả, là công cụ không thể thiếu cho các nhà phân tích dữ liệu.

2.1.2 Cài đặt Pandas

Để cài đặt Pandas, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy, cung cấp một môi trường dễ sử dụng để quản lý các gói và môi trường ảo. Để cài đặt Pandas thông qua Anaconda, bạn có thể sử dụng lệnh sau: *conda install pandas*
- Pip: Pip là trình quản lý gói chuẩn cho Python. Nếu bạn không sử dụng Anaconda, bạn có thể cài đặt Pandas thông qua Pip bằng lệnh sau: *pip install pandas*

Sau khi cài đặt, để sử dụng Pandas chúng ta cần khai báo: `import pandas as pd`. Cách viết ngắn gọn này là tiêu chuẩn phổ biến, giúp mã nguồn dễ đọc và tuân theo các quy ước trong cộng đồng Python. Dù có thể đặt tên khác, nhưng dùng `pd` là lựa chọn tối ưu và được khuyến khích để đọc và tuân theo các quy ước chung trong tài liệu và ví dụ.

2.2 Numpy

2.2.1 Giới thiệu về Numpy

Numpy là thư viện cốt lõi của Python dành cho khoa học máy tính, hỗ trợ xử lý các mảng nhiều chiều với kích thước lớn. Thư viện này cung cấp các hàm tối ưu hóa để thao tác trên các mảng, giúp thực hiện các phép tính nhanh chóng và hiệu quả. NumPy đặc biệt hữu ích trong các bài toán liên quan đến đại số tuyến tính, xử lý tín hiệu, và tính toán ma trận, làm cho việc thao tác và phân tích dữ liệu số trở nên đơn giản và hiệu quả hơn.

2.2.2 Cài đặt Numpy

Để cài đặt NumPy, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy, cung cấp một môi trường dễ sử dụng để quản lý các gói môi trường ảo, bạn có thể tải bằng lệnh sau: `conda install numpy`
- Pip: Pip là trình quản lý gói chuẩn cho Python, bạn có thể tải bằng lệnh sau: `pip install numpy`

Sau khi cài đặt, cần khai báo thư viện NumPy để sử dụng trong mã nguồn Python. Thông thường, NumPy được khai báo gọn lại bằng lệnh `import numpy as np`

2.3 Matplotlib

2.3.1 Giới thiệu về Matplotlib

Để hỗ trợ các suy luận thống kê, trực quan hóa dữ liệu là bước không thể thiếu, và Matplotlib là một công cụ phổ biến trong Python. Đây là thư viện vẽ

đồ thị mạnh mẽ, đặc biệt hữu ích khi kết hợp với Python và NumPy. Trong Matplotlib, module Pyplot thường được sử dụng nhất, cung cấp giao diện thân thiện, tương tự MATLAB nhưng dựa trên Python và hoàn toàn mã nguồn mở. Matplotlib cho phép tạo ra nhiều loại biểu đồ như biểu đồ đường, biểu đồ thanh, biểu đồ phân tán và hơn thế nữa, giúp phân tích dữ liệu hiệu quả và trực quan. Với sự linh hoạt và khả năng tùy chỉnh cao, người dùng dễ dàng tạo các hình ảnh trực quan đáp ứng nhu cầu cụ thể của mình.

2.3.2 Cài đặt Matplotlib

Để cài đặt Matplotlib, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip.

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy cung cấp một môi trường dễ sử dụng để quản lý các gói và môi trường ảo lệnh cài đặt qua Anaconda: *conda install matplotlib*.
- Pip: Pip là trình quản lý gói chuẩn cho Python. Lệnh cài đặt qua Pip: *pip install matplotlib*.

Sau khi cài đặt, bạn có thể bắt đầu sử dụng Matplotlib trong mã nguồn Python bằng cách khai báo: `import matplotlib.pyplot as plt`. Đây là cách viết tắt phổ biến, được cộng đồng Python sử dụng rộng rãi, giúp mã nguồn dễ đọc và tuân theo các quy ước chuẩn trong tài liệu và ví dụ.

2.4 Random Forest

Random Forest là một thuật toán học máy mạnh mẽ thuộc nhóm học máy giám sát (supervised learning) và thường được sử dụng cho các bài toán phân loại và hồi quy. Thuật toán này kết hợp nhiều cây quyết định (decision trees) để tạo thành một "rừng" cây, và kết quả cuối cùng được đưa ra dựa trên kết quả từ tất cả các cây trong rừng.

2.4.1 Cấu trúc của Random Forest

Random Forest là một bộ phân loại học theo phương thức ensemble learning, tức là nó xây dựng nhiều mô hình đơn lẻ (các cây quyết định) và kết hợp chúng lại để đưa ra dự đoán chính xác hơn.

- Cây quyết định (Decision Trees): Mỗi cây quyết định trong Random Forest là một cây riêng biệt được xây dựng bằng cách sử dụng một tập con ngẫu nhiên của dữ liệu và các đặc trưng.
- Quy trình Ensemble: Sau khi các cây quyết định được xây dựng, kết quả của mỗi cây sẽ được lấy (dựa trên đa số phiếu bầu cho phân loại hoặc giá trị trung bình cho hồi quy) để đưa ra dự đoán cuối cùng.

2.4.2 Cách thức hoạt động của Random Forest

a) Tạo ra các cây quyết định:

- Tạo ra một tập hợp các mẫu ngẫu nhiên từ tập huấn luyện bằng cách sử dụng Bootstrap sampling (hay còn gọi là sampling với thay thế).
- Tại mỗi nút trong cây, thay vì thử tất cả các đặc trưng để chia, thuật toán chỉ chọn một tập con ngẫu nhiên của các đặc trưng để tạo sự đa dạng giữa các cây.

b) Dự đoán từ mỗi cây:

- Phân loại: mỗi cây đưa ra một dự đoán phân loại. Random Forest sẽ chọn kết quả dự đoán mà cây quyết định chiếm số phiếu bầu cao nhất.
- Hồi quy: kết quả dự đoán là giá trị trung bình của tất cả các cây trong rừng

2.4.3 Ứng dụng của Random Forest

- Phân loại: ví dụ như phân loại email thành thư rác và không phải thư rác, phân loại khách hàng thành nhóm có thể mua và không thể mua, hoặc phân loại bệnh nhân có nguy cơ mắc bệnh hay không.

- Hồi quy: ví dụ như dự đoán giá trị của bất động sản, giá cổ phiếu, hay giá trị của các biến liên tục khác.
- Phân tích tầm quan trọng đặc trưng: Random Forest có thể được sử dụng để phân tích các yếu tố quan trọng trong tập dữ liệu, giúp xác định các đặc trưng có ảnh hưởng mạnh đến dự đoán.

2.5 Logistic Regression

Logistic Regression là một thuật toán học máy cơ bản và phổ biến thuộc nhóm học máy giám sát (supervised learning). Mặc dù có tên là "hồi quy", nhưng thuật toán này chủ yếu được sử dụng cho các bài toán phân loại nhị phân (binary classification), nơi kết quả dự đoán chỉ nằm trong hai trạng thái (ví dụ: Có bệnh hoặc Không có bệnh).

2.5.1 Cấu trúc của Logistic Regression

Logistic Regression hoạt động dựa trên việc thiết lập mối quan hệ giữa các biến độc lập (các chỉ số sức khỏe) và xác suất xảy ra của một biến phụ thuộc (khả năng mắc bệnh).

- Hàm Tuyến tính: Thuật toán bắt đầu bằng cách tính tổng trọng số của các đặc trưng đầu vào (như BMI, Glucose, tuổi...).
- Hàm Sigmoid: Điểm khác biệt cốt lõi là việc đưa kết quả của hàm tuyến tính qua hàm Sigmoid (S-shape). Hàm này có nhiệm vụ nén mọi giá trị đầu vào vào khoảng từ 0 đến 1, đại diện cho mức xác suất.

2.5.2 Cách thức hoạt động của Logistic Regression

- a) Tính toán xác suất:

- Thuật toán gán cho mỗi đặc trưng đầu vào một trọng số (w) thể hiện mức độ ảnh hưởng của nó. Ví dụ, chỉ số Glucose thường có trọng số cao vì nó ảnh hưởng mạnh đến bệnh tiểu đường.
- Hàm Sigmoid sẽ tính toán dựa trên các trọng số này để trả về một con số xác suất (ví dụ: 0.75 tương đương 75%).

b) Quyết định phân loại:

- Một ngưỡng (thông thường là 0.5) được thiết lập để đưa ra kết luận cuối cùng.
- Nếu xác suất ≥ 0.5 mô hình dự đoán là "Có nguy cơ".
- Nếu xác suất < 0.5 mô hình dự đoán là "Bình thường".

2.5.3 Ứng dụng của Logistic Regression

- Phân loại y khoa: Dự đoán khả năng mắc các bệnh mãn tính như tiểu đường, tim mạch dựa trên các chỉ số xét nghiệm lâm sàng.
- Tín dụng ngân hàng: Đánh giá khả năng một khách hàng có thể trả nợ đúng hạn hay không.
- Phân tích tác động: Giúp xác định các yếu tố nguy cơ hàng đầu (như BMI hay huyết áp) ảnh hưởng như thế nào đến khả năng mắc bệnh thông qua các hệ số hồi quy.

2.5.4 Loss Function

Hàm mất mát trong hồi quy tuyến tính là Mean Squared Error (MSE), tính toán trung bình bình phương sai số giữa các giá trị thực tế và giá trị dự đoán:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2.5.5 Các loại Logistic Regression

- Single Logistic Regression (hồi quy Logistic đơn biến): Khi có chỉ một biến độc lập (feature), mô hình trở thành một đường thẳng trong không gian 2 chiều.
- Multiple Logistic Regression (hồi quy Logistic đa biến): Khi có nhiều biến độc lập, mô hình trở thành một siêu phẳng trong không gian n chiều

CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ

3.1 Mã nguồn tiền xử lý dữ liệu

```
print(df.info())
```

```
print(df.describe())
```

```
print(df.dtypes)
```

Đầu tiên em sử dụng hàm `df.info()` để kiểm tra các feature nào bị thiếu và có kiểu dữ liệu thế nào. Em hiển thị lại hàm `df` và dùng lại hàm `info` để kiểm tra xem dữ liệu. Và hình ảnh dưới là kết quả sau khi đã điền các dữ liệu:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5288 entries, 0 to 5287
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                   5288 non-null   int64
1   gender                               5288 non-null   object
2   pulse_rate                           5288 non-null   int64
3   systolic_bp                         5288 non-null   int64
4   diastolic_bp                        5288 non-null   int64
5   glucose                             5288 non-null   float64
6   height                              5288 non-null   float64
7   weight                              5288 non-null   float64
8   bmi                                 5288 non-null   float64
9   family_diabetes                     5288 non-null   int64
10  hypertensive                        5288 non-null   int64
11  family_hypertension                 5288 non-null   int64
12  cardiovascular_disease              5288 non-null   int64
13  stroke                              5288 non-null   int64
14  diabetic                            5288 non-null   object
dtypes: float64(4), int64(9), object(2)
memory usage: 619.8+ KB
```

Hình 3.1: Thông tin từng cột của Dataset

a) Xử lý thông tin các cột: Height, Diabetic, Gender

```
df = df.drop(df[df['height'] < 1.0].index).reset_index(drop=True)
df = df.dropna().reset_index(drop=True)
df['diabetic'] = df['diabetic'].map({'No': 0, 'Yes': 1})
df['gender'] = df['gender'].map({'Female': 0, 'Male': 1})

selected_cols = ['age', 'glucose', 'bmi', 'systolic_bp', 'diastolic_bp',
                 'family_diabetes', 'hypertensive', 'stroke', 'diabetic']
```

✓ 0.0s

age	gender	pulse_rate	systolic_bp	diastolic_bp	glucose	height	weight	bmi	family_diabetes	hypertensive	family_hypertension
42	0	66	110	73	5.88	1.65	70.2	25.75	0	0	0
35	0	60	125	68	5.71	1.47	42.5	19.58	0	0	0
62	0	57	127	74	6.85	1.52	47.0	20.24	0	0	0
73	1	55	193	112	6.28	1.63	57.4	21.72	0	0	0
68	0	71	150	81	5.71	1.42	36.0	17.79	0	0	0

Hình 3.2: Lọc và mã hóa nhãn

- Đoạn mã chủ yếu lọc và mã hóa nhãn gender và diabetic về dạng số loại bỏ các giá trị không hợp lệ.

b) Xử lý giá trị ngoại lai

```
def handle_outliers_iqr(df, columns):
    df_clean = df.copy()
    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df_clean[col] = df_clean[col].clip(lower=lower_bound, upper=upper_bound)

    return df_clean

cols_to_check = ['age', 'pulse_rate', 'systolic_bp', 'diastolic_bp', 'glucose', 'height', 'weight', 'bmi']
df = handle_outliers_iqr(df, cols_to_check)
print("Outliers handled using IQR method (Capping).")
```

Hình 3.3: Mã nguồn xử lý giá trị ngoại lai

- Sử dụng df.copy() để tránh làm biến đổi dữ liệu gốc
- Tự động phát hiện các giá trị nằm quá xa mức trung bình (nằm ngoài khoảng)

c) Kiểm tra giá trị thiếu

```
print("\nMissing values:")
print(df.isnull().sum())
```

✓ 0.0s

```
# Kiểm tra tổng số giá trị thiếu ở mỗi cột
missing_info = df.isnull().sum()
print("Số lượng giá trị thiếu ở mỗi cột:")
print(missing_info[missing_info > 0])

# Liệt kê các cột số cần điền giá trị
cols_to_fix = ['age', 'pulse_rate', 'systolic_bp', 'diastolic_bp', 'glucose', 'height', 'weight', 'bmi']

# Điền giá trị thiếu bằng trung vị của cột đó
for col in cols_to_fix:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())
print("Đã xử lý xong giá trị thiếu bằng phương pháp điền trung vị.")
```

Hình 3.4: Kiểm tra & xử lý giá trị thiếu

- Đếm số lần xuất hiện và cộng tổng số lượng các giá trị thiếu bằng hàm `isnull().sum()`
- Liệt kê ra các cột chứa giá trị thiếu sau đó điền cho các giá trị thiếu bằng một giá trị trung vị bằng hàm `median()`

3.2 Mã nguồn thống kê dữ liệu

a) Thống kê số lượng bệnh nhân nam và nữ

```
df['gender'].value_counts().head(5)
```

✓ 0.0s

```
gender
0    3737
1    1532
Name: count, dtype: int64
```

Hình 3.5: Mã nguồn đếm số lượng giới tính

- Đếm số lượng bệnh nhân nam và nữ trong cột 'Gender': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'Gender'

b) Thống kê số lượng bệnh nhân mắc bệnh tiểu đường trong dữ liệu

```
df['diabetic'].value_counts().head(5)
```

✓ 0.0s

```
diabetic
0      4932
1       337
Name: count, dtype: int64
```

Hình 3.6: Mã nguồn đếm số lượng mắc bệnh

- Đếm tần xuất giá trị trong cột 'Diabetic' sử dụng phương thức `value_counts()` để đếm số lần xuất hiện các giá trị trong cột 'Diabetic'

c) Thống kê số người có mức glucose cao

```
glucose_mean = df['glucose'].mean()

high_glucose_df = df[df['glucose'] > glucose_mean]
num_high_glucose = len(high_glucose_df)

total_people = len(df)
percentage = (num_high_glucose / total_people) * 100

print(f"Mức Glucose trung bình: {glucose_mean:.2f} mmol/L")
print(f"Số lượng người có mức Glucose cao hơn trung bình: {num_high_glucose} người")
print(f"Tỷ lệ: {percentage:.2f}% trên tổng số dữ liệu")
print("\nThống kê tình trạng bệnh trong nhóm Glucose cao:")
print(high_glucose_df['diabetic'].value_counts())
```

✓ 0.0s

Mức Glucose trung bình: 7.51 mmol/L
Số lượng người có mức Glucose cao hơn trung bình: 1922 người
Tỷ lệ: 36.48% trên tổng số dữ liệu

Thống kê tình trạng bệnh trong nhóm Glucose cao:

```
diabetic
0      1697
1       225
```

Hình 3.7: Mã nguồn đếm số lượng người có mức glucose cao

- Đếm số lượng người có mức đường huyết 'Glucose' cao hơn mức trung bình 7.51 mol/l so sánh với glicose_mean để biết tỉ lệ người mắc bệnh khi có mức đường huyết cao hơn mức trung bình.

d) Thống kê số người có mức bmi cao

```
bmi_mean = df['bmi'].mean()
# Lọc ra những người có chỉ số BMI cao hơn mức trung bình
high_bmi_df = df[df['bmi'] > bmi_mean]

num_high_bmi = len(high_bmi_df)
total_people = len(df)
percentage = (num_high_bmi / total_people) * 100

print(f"Trung bình BMI: {bmi_mean:.2f}")
print(f"Số người có mức BMI > trung bình: {num_high_bmi}")
print(f"Phần trăm: {percentage:.2f}% của tổng dữ liệu")
print("\nDiabetic số người :")
print(high_bmi_df['diabetic'].value_counts())
```

✓ 0.0s

Trung bình BMI: 22.34
Số người có mức BMI > trung bình: 2383
Phần trăm: 45.23% của tổng dữ liệu

Diabetic số người :
diabetic
0 2180
1 203

Hình 3.8: Mã nguồn đếm số lượng người có mức bmi cao

- Đếm số lượng người có mật độ khối trong cơ thể cao hơn mức trung bình ta có thể biết rõ hơn những người có mật độ khối trong cơ thể cao hơn thì tỉ lệ mắc bệnh tiểu đường của họ có cao hơn không.

3.3 Mã nguồn trực quan hóa dữ liệu

1. Biểu đồ histplot cho phân bố mức Glucose cao với số lượng người trong dữ liệu

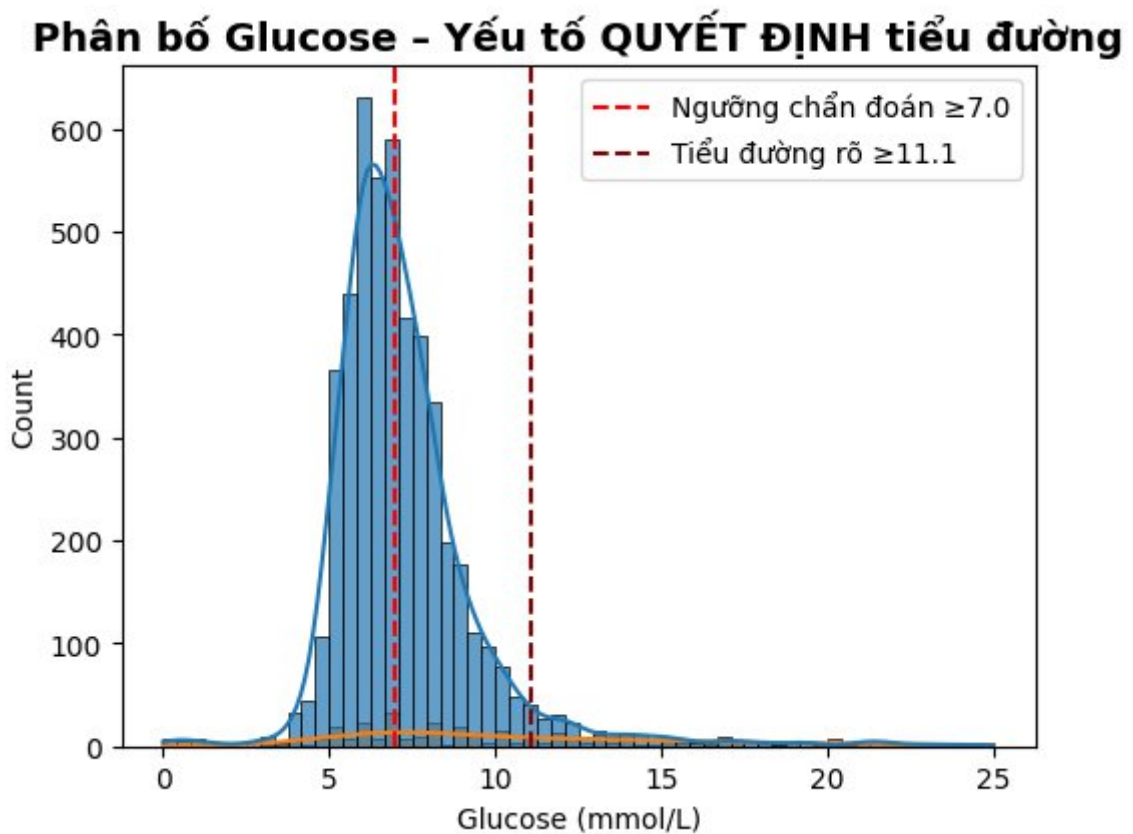
```
fig = plt.figure(figsize=(20, 15))

plt.subplot(3, 3, 1)
sns.histplot(data=df, x='glucose', hue='diabetic', bins=60, kde=True, alpha=0.7)
plt.title('Phân bố Glucose ☐ Yếu tố QUYẾT ĐỊNH tiểu đường', fontsize=14, fontweight='bold')
plt.xlabel('Glucose (mmol/L)')
plt.axvline(7.0, color='red', linestyle='--', label='Ngưỡng chẩn đoán ≥7.0')
plt.axvline(11.1, color='darkred', linestyle='--', label='Tiểu đường rõ ≥11.1')
plt.legend()

✓ 0.3s
```

Hình 3.9: Mã nguồn biểu đồ histplot phân bố mức glucose

- Vẽ histplot: sử dụng thư viện Seaborn (sns) để vẽ, giúp hiển thị phân bố mức glucose theo số lượng người trong dữ liệu.
 - plt.figure(figsize = 20, 15) để thiết lập kích thước biểu đồ
 - plt.subplot(3,3,1) để chia biểu đồ thành lưới 3 hàng 3 cột
 - sns.histplot sử dụng tập dữ liệu
- Tùy chỉnh biểu đồ plt.title() để đặt tiêu đề biểu đồ
 - plt.xlabel() để gán 'Glucose' lên trục hoành
 - plt.axvline() để kẻ ngưỡng dữ liệu



Hình 3.10: Biểu đồ histplot hiển thị phân bố glucose

2. Biểu đồ hiển thị mức độ tương quan

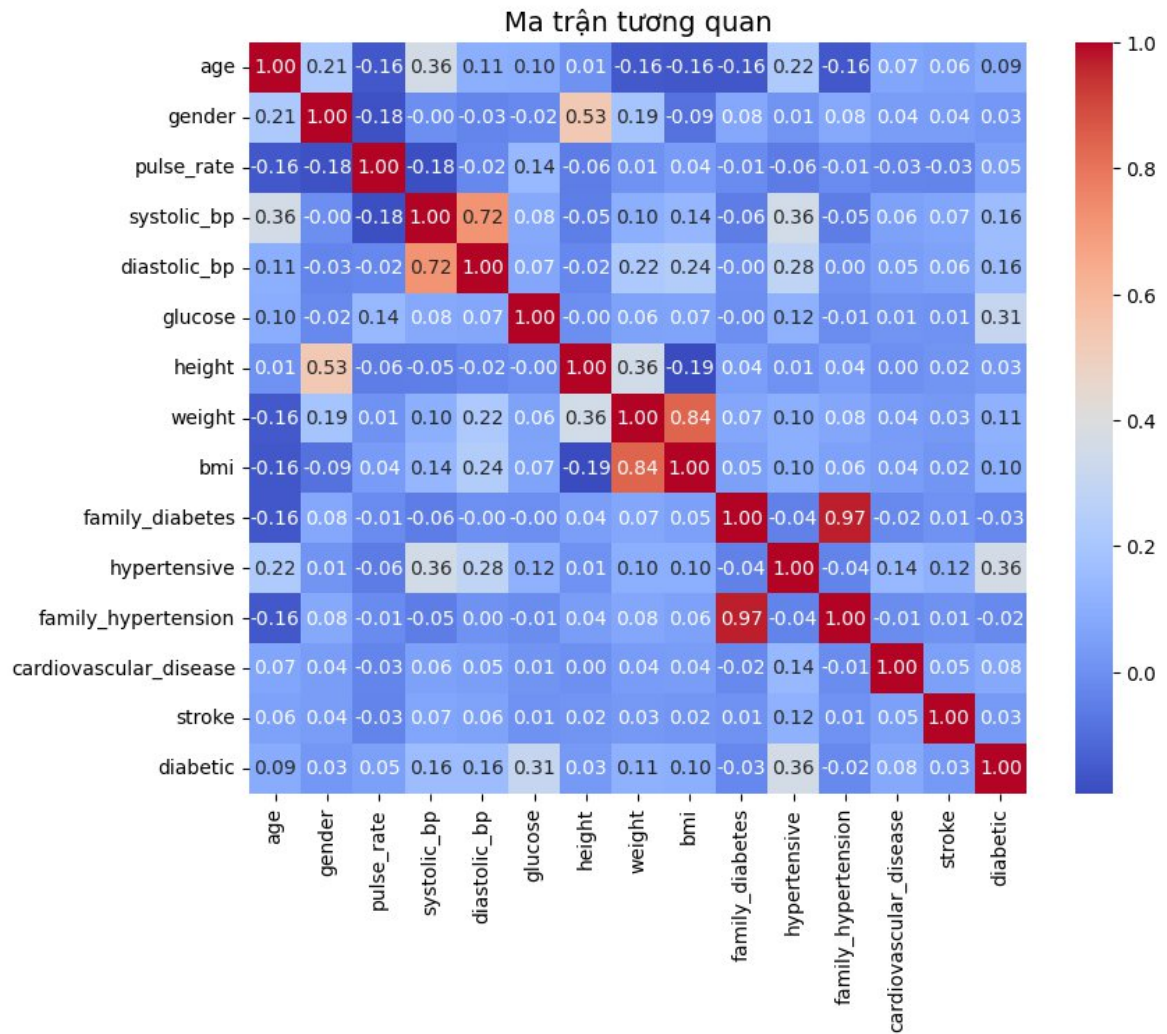
```
fig = plt.figure(figsize=(20, 15))
plt.figure(figsize=(9, 7))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Ma trận tương quan', fontsize=14)
plt.show()
```

✓ 0.2s

Hình 3.11: Mã nguồn biểu đồ heatmap

- Vẽ biểu đồ heatmap: sử dụng Seaborn để vẽ
 - `plt.figure(figsize)` để thiết lập kích thước biểu đồ
 - `sns.heatmap()` sử dụng `df.corr()` để tính toán độ tương quan giá trị chạy từ -1 đến 1 càng gần 1 thì càng tương quan mật thiết đến nhau, `annot=True` để điền số cụ thể vào ô vuông, `cmap='coolwarm'` thiết lập bảng màu, `fmt='.2f'` để làm tròn đến chữ số thập phân thứ 2

- plt.title() để điền tiêu đề cho biểu đồ
- plt.show() để hiển thị biểu đồ



Hình 3.12: Biểu đồ tương quan dữ liệu

3. Biểu đồ bars để hiển thị mức độ mắc bệnh theo nhóm tuổi

```
def nhom_tuoi(age):
    if age < 40:
        return "Dưới 40 tuổi"
    elif age < 60:
        return "40 - 59 tuổi"
    else:
        return "60 tuổi trở lên"
df['nhom_tuoi'] = df['age'].apply(nhom_tuoi)

ty_le = df.groupby('nhom_tuoi')['diabetic'].mean() * 100
order = ["Dưới 40 tuổi", "40 - 59 tuổi", "60 tuổi trở lên"]
ty_le = ty_le.reindex(order)

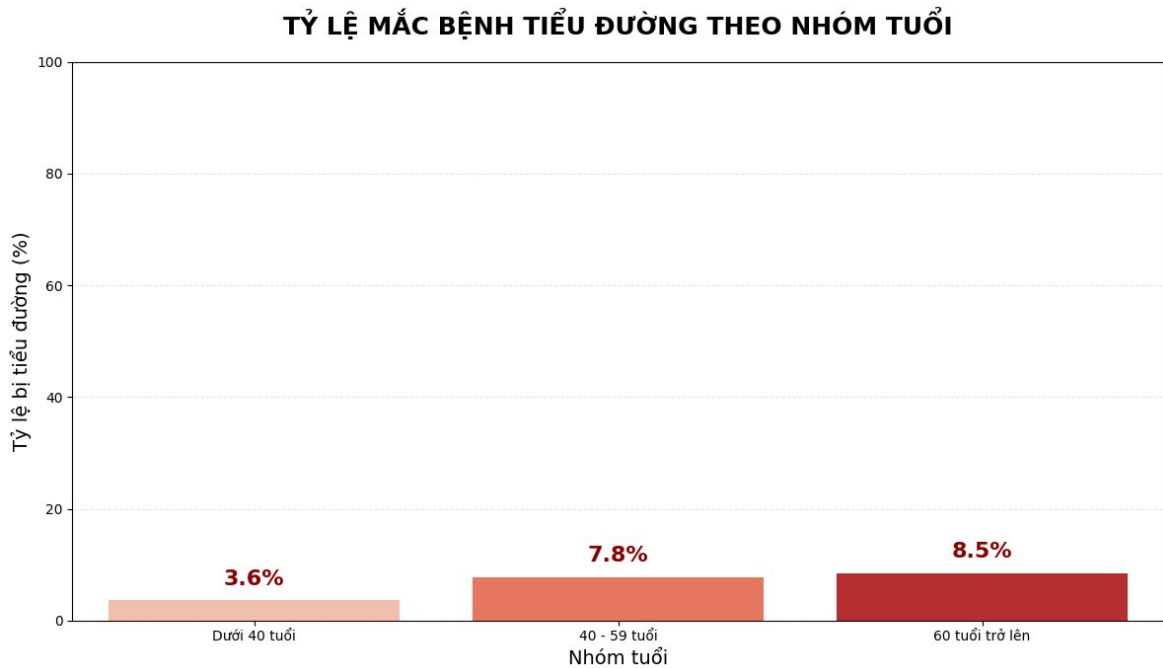
plt.figure(figsize=(12, 7))
bars = sns.barplot(x=ty_le.index, y=ty_le.values, palette="Reds")
plt.title('TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO NHÓM TUỔI',
          fontsize=18, fontweight='bold', pad=20)
plt.ylabel('Tỷ lệ bị tiểu đường (%)', fontsize=14)
plt.xlabel('Nhóm tuổi', fontsize=14)
plt.ylim(0, 100)

for i, v in enumerate(ty_le.values):
    plt.text(i, v + 2, f'{v:.1f}%', ha='center', va='bottom',
            fontsize=16, fontweight='bold', color='darkred')
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.tight_layout()
plt.show()
```

Hình 3.13: Mã nguồn vẽ biểu đồ mắc bệnh theo nhóm tuổi

- Sử dụng hàm `nhom_tuoi` để phân nhóm tuổi ra làm ba nhóm: dưới 40 tuổi, từ 40 đến 60 tuổi và trên 60 tuổi, nhân `mean() * 100` để lấy tỉ lệ phân trăm
- Vẽ biểu đồ bars sử dụng Seaborn (`sns`) để vẽ biểu đồ `barplot` giúp hiển thị tỉ lệ mắc bệnh theo từng nhóm tuổi
 - `plt.figure(figsize=(12, 7))` để thiết lập kích thước biểu đồ
 - `sns.barplot(x=ty_le.index, y=ty_le.values, palette='Reds')` để khởi tạo một cửa sổ biểu đồ với màu đỏ
- Tùy chỉnh biểu đồ: `plt.title()` đặt tên cho biểu đồ
 - `plt.ylim(0, 100)` thiết lập phạm vi trục tung đến 100%
 - `plt.grid(axis='y', alpha=0.3, linestyle='--')` để vẽ dòng kẻ ngang và độ trong suốt đường kẻ

- `plt.tight_layout()` để tự động điều chỉnh các thành phần không bị đè lên nhau



Hình 3.14: Biểu đồ bars hiển thị mức độ mắc bệnh theo nhóm tuổi

4. Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo giới tính

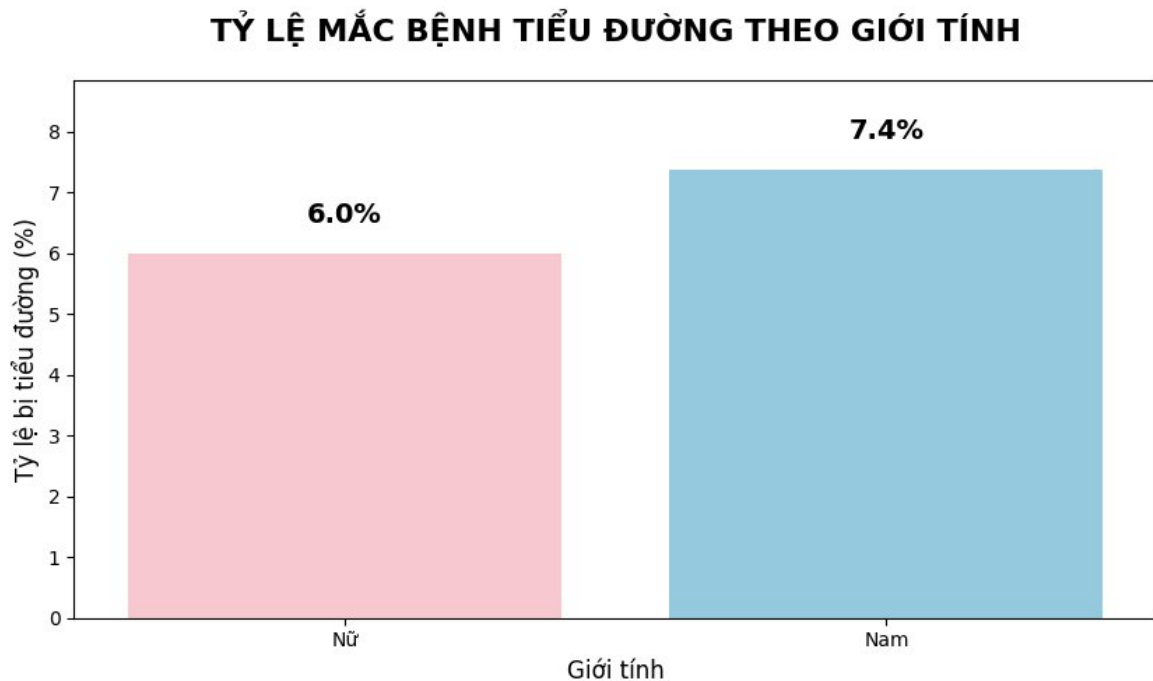
```
rate = df.groupby('gender')['diabetic'].mean() * 100
plt.figure(figsize=(10,5))
sns.barplot(x=['Nữ', 'Nam'], y=rate.values, palette=['pink', 'skyblue'])
plt.title('TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO GIỚI TÍNH', fontsize=16, fontweight='bold', pad=20)
plt.ylabel('Tỷ lệ bị tiểu đường (%)', fontsize=12)
plt.xlabel('Giới tính', fontsize=12)

for i, v in enumerate(rate.values):
    plt.text(i, v + 0.5, f'{v:.1f}%', ha='center', fontweight='bold', fontsize=14)
plt.ylim(0, max(rate)*1.2)
plt.show()
```

Hình 3.15: Mã nguồn biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo giới tính

- Tính `mean()` của cột 'diabetic' theo cột 'gender' sau đó $\times 100$ để lấy tỉ lệ phần trăm hiển thị
- Vẽ biểu đồ barplot để hiển thị số lượng bệnh nhân theo giới tính
 - `plt.figure(figsize=(10,5))` thiết lập kích thước biểu đồ
 - `sns.barplot(x=['nữ', 'nam'])` gán tên và chọn màu cho hai cột

- Tùy chỉnh biểu đồ: plt.title() đặt tên cho biểu đồ
 - plt.ylabel() tên trục tung
 - plt.xlabel() tên trục hoành
- Sử dụng vòng lặp để tính tỉ lệ mắc bệnh theo giới tính



Hình 3.16: Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo giới tính

5. Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo tiền sử gia đình

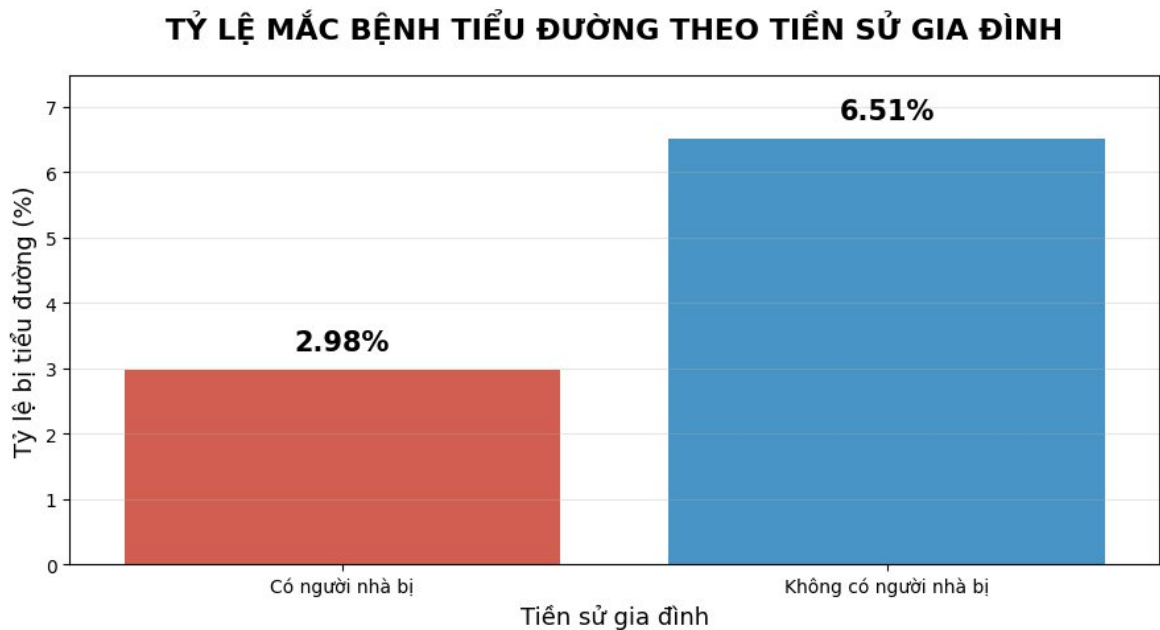
```
rate = df.groupby('family_diabetes')['diabetic'].mean() * 100
# Sắp xếp đúng thứ tự: 1 (Có) trước, 0 (Không) sau
rate = rate.reindex([1, 0])
plt.figure(figsize=(9, 5))
bars = sns.barplot(x=['Có người nhà bị', 'Không có người nhà bị'],
                  y=rate.values,
                  palette=[ '#e74c3c', '#3498db' ])
plt.title('TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO TIỀN SỬ GIA ĐÌNH',
          fontsize=16, fontweight='bold', pad=20)
plt.ylabel('Tỷ lệ bị tiểu đường (%)', fontsize=13)
plt.xlabel('Tiền sử gia đình', fontsize=13)

for i, v in enumerate(rate.values):
    plt.text(i, v + 0.3, f'{v:.2f}%', ha='center', fontweight='bold', fontsize=15)

plt.ylim(0, max(rate.values)*1.15)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

Hình 3.17: Mã nguồn barplot hiển thị tỉ lệ mắc bệnh theo gia đình

- rate để tính phần trăm của cột family_diabetic sử dụng hàm mean() * 100 để thực hiện
- Vẽ biểu đồ barplot để hiển thị tỉ lệ mắc bệnh tiểu đường theo tiền sử gia đình
 - plt.figure(figsize=(9, 5)) để thiết lập kích thước biểu đồ
 - sns.barplot() sử dụng Seaborn để vẽ, x đặt tên cho các cột và thiết lập màu sắc bằng palette.
- Tùy chỉnh biểu đồ: plt.title() để đặt tên cho biểu đồ
 - plt.ylabel tiêu đề trục tung
 - plt.xlabel tiêu đề trục hoành
- Vòng lặp for để tính toán tỉ lệ phần trăm tỉ lệ mắc bệnh theo cột family_diabetic
 - plt.ylim thiết lập phạm vi trục y
 - plt.grid() thiết lập lưới cho biểu đồ và độ dài đường lưới
 - plt.tight_layout() để tự động điều chỉnh khoảng cách giữa các thành phần



Hình 3.18: Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo gia đình

6. Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo tiền sử đột quỵ

```
rate = df.groupby('stroke')['diabetic'].mean() * 100
rate = rate.reindex([1, 0])

plt.figure(figsize=(9, 6))
bars = sns.barplot(
    x=['Có tiền sử đột quỵ', 'Không có đột quỵ'],
    y=rate.values,
    palette=[ '#d63031', '#00b894' ] # Đỏ = nguy hiểm, Xanh = an toàn
)
plt.title('TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO TIỀN SỬ ĐỘT QUỴ',
          fontsize=16, fontweight='bold', pad=25, color='darkred')
plt.ylabel('Tỷ lệ bị tiểu đường (%)', fontsize=13)
plt.xlabel('Tiền sử đột quỵ', fontsize=13)

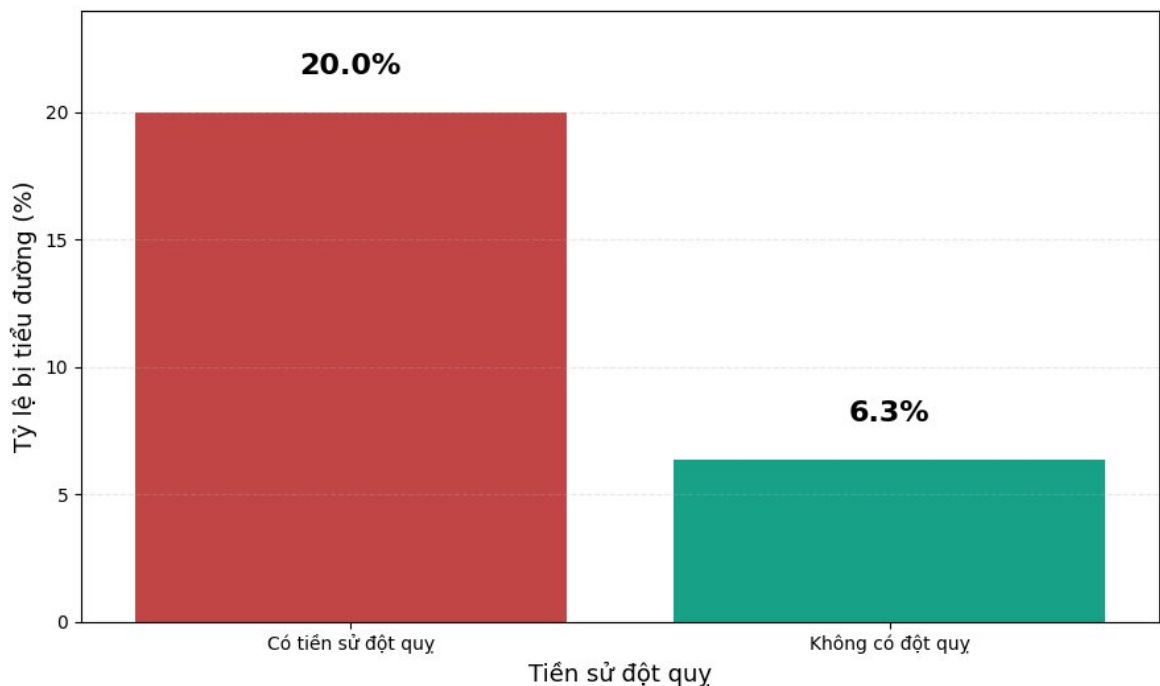
for i, v in enumerate(rate.values):
    plt.text(i, v + 1.5, f'{v:.1f}%', ha='center', fontweight='bold', fontsize=16)
plt.ylim(0, max(rate.values)*1.2)
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.tight_layout()
plt.show()
```

Hình 3.19: Mã nguồn barplot hiển thị tỉ lệ mắc bệnh theo đột quỵ

- Gom nhóm tập dữ liệu thành hai loại trong cột stroke, tính giá trị trung bình của cột diabetic chuyển tỉ lệ đó thành phần trăm khi nhân `mean() * 100`

- Vẽ biểu đồ barplot hiển thị tỉ lệ người mắc bệnh khi có tiền sử bị đột quỵ
 - `plt.figure(figsize=(9, 6))` để thiết lập kích thước biểu đồ
 - `sns.barplot` sử dụng Seaborn để vẽ biểu đồ, đặt tên cho các cột và thiết lập màu sắc bằng palette
- Tùy chỉnh biểu đồ: `plt.title()` để đặt tên cho biểu đồ
 - `plt.ylabel` tiêu đề trục tung
 - `plt.xlabel` tiêu đề trục hoành
- Vòng lặp for để tính toán tỉ lệ phần trăm tỉ lệ mắc bệnh theo cột stroke
 - `plt.ylim` thiết lập phạm vi trục y
 - `plt.grid()` thiết lập lưới cho biểu đồ và độ dài đường lưới
 - `plt.tight_layout()` để tự động điều chỉnh khoảng cách giữa các thành phần

TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO TIỀN SỬ ĐỘT QUỴ



Hình 3.20: Biểu đồ barplot hiển thị tỉ lệ mắc bệnh theo đột quỵ

7. Biểu đồ figure mắc bệnh tiểu đường theo nhóm đường huyết

```
labels = ['Đường huyết CAO ( $\geq 7.0$ )', 'Đường huyết BÌNH THƯỜNG ( $< 7.0$ )']
sizes = rate.values
colors = ['#e74c3c', '#2ecc71']
explode = (0.1, 0)

plt.figure(figsize=(8, 8))
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=140,
        textprops={'fontsize': 14, 'fontweight': 'bold'},
        wedgeprops={'width': 0.4, 'edgecolor': 'w'})

plt.title('TỶ LỆ NGUY CƠ TIỂU ĐƯỜNG THEO NHÓM ĐƯỜNG HUYẾT',
        fontsize=16, fontweight='bold', color='black', pad=20)
plt.tight_layout()
plt.show()
```

✓ 0.1s

Hình 3.21: Mã nguồn biểu đồ nguy cơ tiểu đường theo đường huyết

- Sử dụng hai label so sánh, sử dụng hai màu và `explode(0.1, 0)` làm nổi bật nguy cơ mắc bệnh
- Vẽ biểu đồ figure hiển thị nguy cơ mắc bệnh tiểu đường theo mức đường huyết
 - `plt.figure(figsize=(8,8))` thiết lập kích thước biểu đồ
 - `plt.pie(sizes, explode=explode, label=label, colors=colors, autopct='%1.1f%%', shadow=true, startangle=140)` sử dụng để gán các giá trị số tương ứng với từng phần biểu đồ, label tên văn bản gán cho mỗi phần, color danh sách các màu sử dụng cho mỗi phần, các hiệu ứng `explode` làm nổi bật biểu đồ, thêm phần đổ bóng `shadow=True`, cuối cùng hiển thị phần trăm cho mỗi phần chỉ một chữ số sau dấu phẩy.
- Tùy chỉnh biểu đồ: `plt.title()` để đặt tên cho biểu đồ
 - `plt.tight_layout()` tự động điều chỉnh khoảng cách sau mỗi phần

TỶ LỆ NGUY CƠ TIỂU ĐƯỜNG THEO NHÓM ĐƯỜNG HUYẾT



Hình 3.22: Biểu đồ nguy cơ tiểu đường theo đường huyết

8. Biểu đồ tỉ lệ mắc bệnh tiểu đường theo nhóm BMI

```
def phan_nhom_bmi(bmi):
    if bmi < 18.5: return "Gầy"
    elif bmi < 23: return "Bình thường"
    elif bmi < 25: return "Thừa cân"
    elif bmi < 30: return "Béo phì độ 1"
    else: return "Béo phì độ 2+"

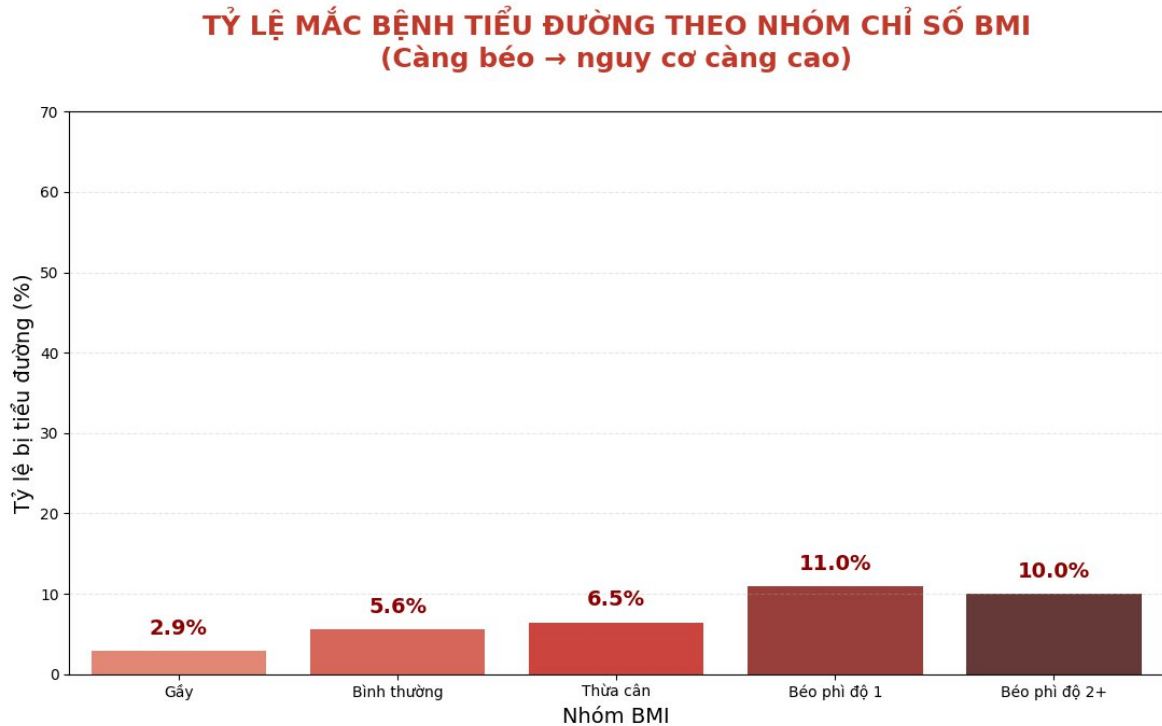
df['bmi_group'] = df['bmi'].apply(phan_nhom_bmi)
rate = df.groupby('bmi_group')['diabetic'].mean() * 100
order = ["Gầy", "Bình thường", "Thừa cân", "Béo phì độ 1", "Béo phì độ 2+"]
rate = rate.reindex(order)
plt.figure(figsize=(11, 7))
bars = sns.barplot(x=rate.index, y=rate.values,
                  palette="Reds_d")
plt.title('TỶ LỆ MẮC BỆNH TIỂU ĐƯỜNG THEO NHÓM CHỈ SỐ BMI\n(Càng béo → nguy cơ càng cao)',
          fontsize=18, fontweight='bold', pad=30, color='c0392b')
plt.ylabel('Tỷ lệ bị tiểu đường (%)', fontsize=14)
plt.xlabel('Nhóm BMI', fontsize=14)
plt.ylim(0, 70)

for i, v in enumerate(rate.values):
    plt.text(i, v + 1.5, f'{v:.1f}%', ha='center', va='bottom',
            fontsize=14, fontweight='bold', color='darkred')
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.tight_layout()
✓ 0.2s
```

Hình 3.23: Mã nguồn biểu đồ nguy cơ tiểu đường theo bmi

- Sử dụng hàm `def phan_nhom_bmi` để phân ra các nhóm khối trong cơ thể người ra làm các mức độ: gầy, bình thường, thừa cân, béo phì 1, béo phì 2
- Tạo cột mới từ dữ liệu đã được phân chia mức độ tên là `'bmi_group'` sau đó gom nhóm dữ liệu cột `'bmi_group'` sau đó tính tỉ lệ người mắc bệnh
- Sử dụng biểu đồ `barplot` thực hiện vẽ biểu đồ hiển thị nguy cơ mắc bệnh tiểu đường theo chỉ số khối bmi
 - `plt.figure(figsize=(11,7))` để thiết lập kích cỡ biểu đồ
 - `sns.barplot` để thực hiện vẽ biểu đồ với `rate_index` và `rate_value` là các giá trị đã được định nghĩa trên `order` và tỉ lệ phần trăm
- Tùy chỉnh biểu đồ: `plt.title()` để đặt tên cho biểu đồ
 - `plt.ylabel` tiêu đề trục tung

- plt.xlabel tiêu đề trục hoành
- Sử dụng For để tính toán cụ thể tỉ lệ phần trăm mắc bệnh theo bmi
 - Sử dụng plt.grid() để vẽ lưới cho biểu đồ
 - plt.tight_layout() để tự động điều chỉnh khoảng cách giữa các vật thể



Hình 3.24: Biểu đồ nguy cơ tiểu đường theo bmi

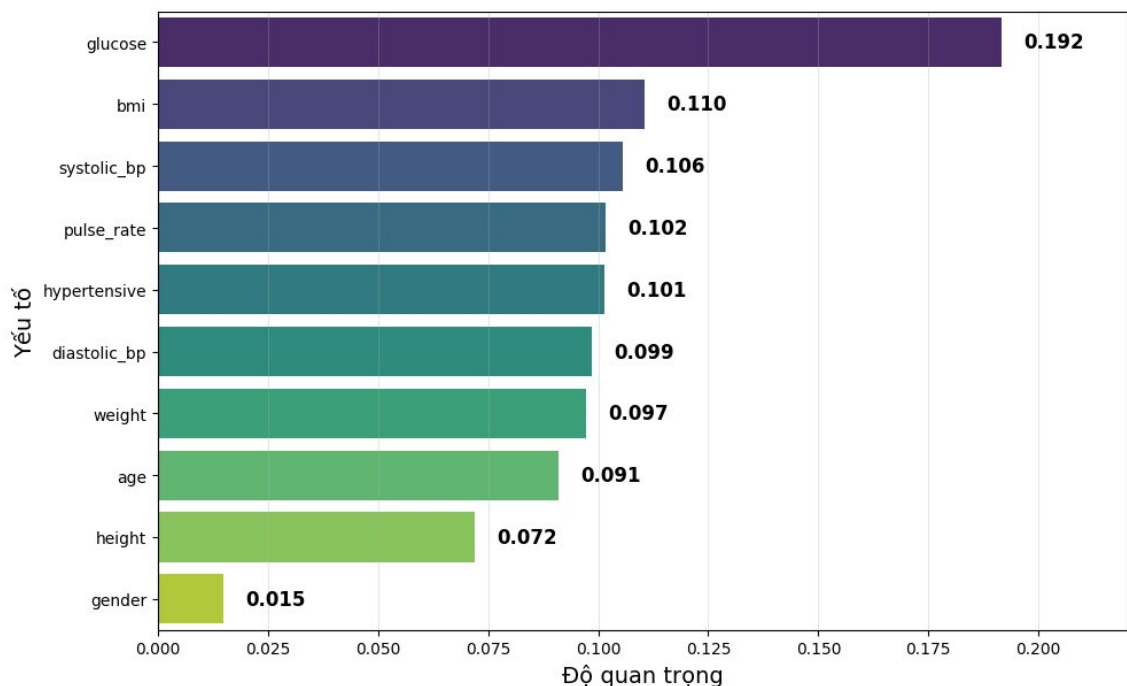
9. Biểu đồ các feature quan trọng để dự đoán nguy cơ mắc bệnh tiểu đường



Hình 3.25: Mã nguồn biểu đồ các yếu tố quan trọng

- Sử dụng biểu đồ barplot để vẽ biểu đồ hiển thị các yếu tố quan trọng để dự đoán nguy cơ mắc bệnh tiểu đường
 - `plt.figure(figsize=(10,7))` thiết lập kích thước cho biểu đồ
 - `sns.barplot(x=feat_imp.value[:10], y=feat_imp.index[:10], palette='viridis')` sử dụng để lấy ra mười các đặc trưng quan trọng nhất và hiển thị cả tên của các đặc trưng, sử dụng palette để thiết lập màu sắc riêng biệt.
- Tùy chỉnh biểu đồ: `plt.title()` để đặt tên cho biểu đồ
 - `plt.ylabel` tiêu đề trục tung
 - `plt.xlabel` tiêu đề trục hoành
- Sử dụng For để thêm các con số và cạnh phải các feature được vẽ hiển thị độ quan trọng
 - `plt.xlim(0, max(feat_imp.values[:10]) * 1.15)` để tìm giá trị số quan trọng nhất.
 - Sử dụng `plt.grid()` để vẽ lưới cho biểu đồ
 - `plt.tight_layout()` để tự động điều chỉnh khoảng cách giữa các vật thể

TOP 10 YẾU TỐ QUAN TRỌNG NHẤT DỰ ĐOÁN BỆNH TIỂU ĐƯỜNG (theo Random Forest)



Hình 3.26: Biểu đồ các feature quan trọng

10. Biểu đồ hiển thị tỉ lệ mắc bệnh tiểu đường trong dữ liệu

```
fig = plt.figure(figsize=(20, 15))
plt.subplot(3, 3, 9)
sizes = df['diabetic'].value_counts()
plt.pie(sizes, labels=['Không bị', 'Bị tiểu đường'], autopct='%1.1f%%', colors=[ '#66b3ff', '#ff9999'], startangle=90)
plt.title('Tỷ lệ bị tiểu đường trong dữ liệu', fontweight='bold')

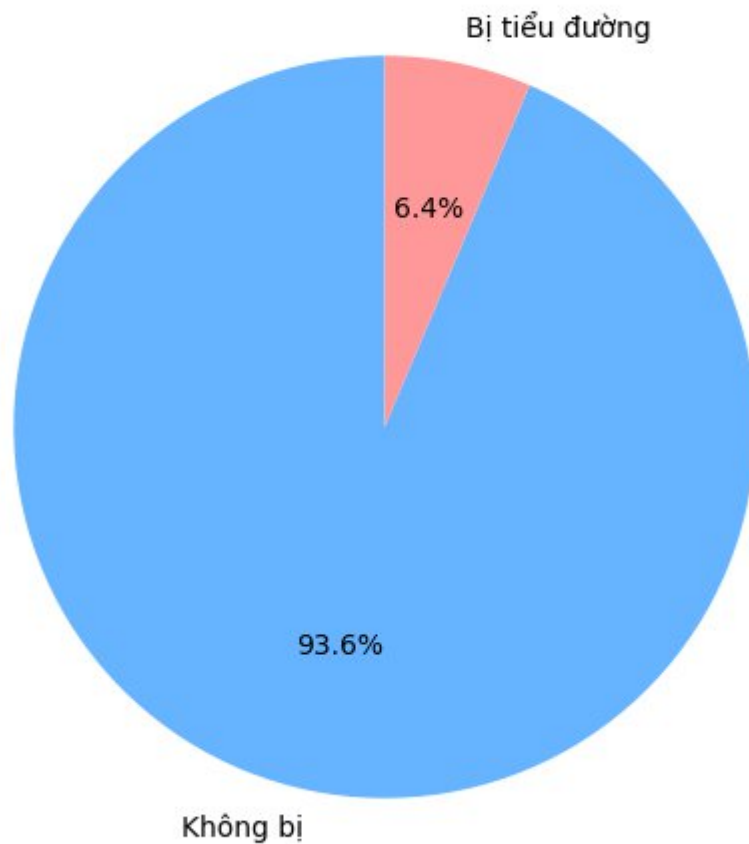
plt.tight_layout()
plt.show()
```

✓ 0.0s

Hình 3.27: Mã nguồn biểu đồ mắc bệnh tiểu đường

- Sử dụng biểu đồ subplot để vẽ biểu đồ hiển thị
 - `plt.figure(figsize=(20,15))` để thiết lập kích thước biểu đồ
- Đếm số lượng mắc bệnh tiểu đường trong dữ liệu cột 'Diabetic' bằng hàm `value_counts()`
 - `plt.pie` thiết lập tên các vùng dữ liệu lấy tỉ lệ phần trăm một số thập phân
 - `plt.title()` đặt tên cho biểu đồ
 - `plt.tight_layout()` để tự động điều chỉnh khoảng cách giữa các phần tử

Tỷ lệ bị tiểu đường trong dữ liệu



Hình 3.28: Biểu đồ mắc bệnh tiểu đường trong dữ liệu

3.4 Áp dụng model

```
df_model = df.select_dtypes(include=['number'])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

✓ 0.0s

Hình 3.29: Chia dataset ra làm hai phần để train và test

- Chia dữ liệu dữ liệu thành tập huấn luyện (train) và tập kiểm tra (test):
 - X là các đặc trưng (feature) và y là biến mục tiêu (target)

- Test_size=0.2 là lấy 20% dữ liệu sử dụng cho tập kiểm tra còn 80% dữ liệu còn lại sẽ được sử dụng cho tập huấn luyện
- Random_state = 42 đảm bảo rằng quá trình chia dữ liệu sẽ được lặp lại một cách nhất quán mỗi lần chạy mã, giúp ta có thể tái tạo lại kết quả

```
print(f"\nTrain: {X_train.shape[0]} mẫu, Test: {X_test.shape[0]} mẫu")
print(f"Tỷ lệ dương tính trong train: {y_train.mean():.3f}, test: {y_test.mean():.3f}")
✓ 0.0s
```

Train: 4215 mẫu, Test: 1054 mẫu
Tỷ lệ dương tính trong train: 0.064, test: 0.064

Hình 3.30: Kết quả khi chia dataset ra làm hai tập

❖ Chuẩn hóa :

```
scaler = StandardScaler()
cols_to_scale = ['age', 'pulse_rate', 'systolic_bp', 'diastolic_bp', 'glucose',
                 'height', 'weight', 'bmi']

X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])
X_test_scaled[cols_to_scale] = scaler.transform(X_test[cols_to_scale])
✓ 0.0s
```

Hình 3.31: Chuẩn hóa dữ liệu

- Chuẩn hóa là quá trình đưa các thuộc tính dữ liệu có các đơn vị và khoảng giá trị khác nhau về cùng một hệ quy chiếu. Tại sao cần chuẩn hóa :
 - Tránh sự chênh lệch đơn vị giữa giá trị nhỏ nhất và lớn nhất
 - Tăng sự hội tụ giúp hàm tìm kết quả nhanh hơn
 - Giúp công bằng giữa các đặc trưng

❖ Smote dữ liệu :

```
smote = SMOTE(random_state=42)
# sinh dữ liệu
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
print("Sau khi SMOTE:", y_train_smote.value_counts())
```

✓ 0.0s

Hình 3.32: Smote dữ liệu

- Smote dữ liệu là một công cụ mạnh mẽ giúp khắc phục tình trạng mất cân bằng dữ liệu. Trong nhiều tình huống thực tế, như phát hiện bệnh tiểu đường, số lượng người "Khỏe mạnh" thường lớn hơn nhiều so với số lượng người "Bị tiểu đường". Nếu mô hình huấn luyện trên dữ liệu như vậy, nó có thể bị thiên kiến về phía nhóm đa số. SMOTE tạo ra các mẫu "tổng hợp" (giả nhưng thực tế) cho nhóm thiểu số để cân bằng lại cán cân dữ liệu.
- Sử dụng `random_state = 42` để đảm bảo quá trình smote sẽ được lặp lại một cách nhất quán sau mỗi lần chạy mã.

```
Sau khi SMOTE: diabetic
0      3945
1      3945
Name: count, dtype: int64
```

Hình 3.33: kết quả sau khi smote dữ liệu

❖ Huấn luyện mô hình :

```
def evaluate_clf(model, model_name, X_test, y_test):
    """
    Hàm đánh giá mô hình phân loại.
    Trả về: accuracy, precision, recall, f1 + vẽ 2 biểu đồ quan trọng.
    (Đã loại bỏ ROC AUC và ROC Curve theo yêu cầu)
    """

    y_pred = model.predict(X_test)

    # Lấy xác suất dự đoán (chỉ dùng cho Precision-Recall Curve)
    if hasattr(model, "predict_proba"):
        y_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_proba = model.decision_function(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
```

```
print(f"\n===== {model_name} =====")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall : {rec:.4f}")
print(f"F1-score : {f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Chỉ vẽ 2 biểu đồ
fig, axs = plt.subplots(1, 2, figsize=(12, 4))

PrecisionRecallDisplay.from_predictions(y_test, y_proba, ax=axs[0])
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=axs[1])

axs[0].set_title("Precision-Recall Curve")
axs[1].set_title("Confusion Matrix")

plt.tight_layout()
plt.show()
```

```
# Lưu vào bảng kết quả (không có roc_auc)
return {
    "modelname": model_name,
    "accuracy": acc,
    "precision": prec,
    "recall": rec,
    "f1": f1
}

# Cập nhật DataFrame để phù hợp
model_metrics = pd.DataFrame(columns=[
    "modelname", "accuracy", "precision", "recall", "f1"
])
```

✓ 0.0s

Hình 3.34: Mã nguồn huấn luyện mô hình

- Hàm `evaluate_clf` là hàm toàn diện để đo lường mức độ hiệu quả của mô hình học máy trong việc dự đoán tiểu đường. Thay vì chỉ nhìn vào một con số, nó xem xét kết quả từ nhiều góc độ khác nhau.
- Xử dụng bốn metric đánh giá:
 - Accuracy: Tỷ lệ dự đoán đúng trên tổng số mẫu.
 - Precision: Trong số những người mô hình đoán bị bệnh, có bao nhiêu người thực sự bị bệnh?
 - Recall: Trong số tất cả những người thực sự bị bệnh, mô hình đã tìm ra được bao nhiêu người? Đây là chỉ số quan trọng nhất trong y tế!
 - F1-score: Sự kết hợp cân bằng giữa Precision và Recall.
- Hàm `y_pred` xuất ra câu trả lời cuối cùng 0 – 1.
- Hàm `y_proba` là mức độ tự tin số này dùng để vẽ các đường cong đánh giá.
- Sau đó vẽ các biểu đồ trực quan rồi lưu kết quả lại rồi hiển thị.

3.4.1 Model Logistic Regression

- Trước khi smote dữ liệu :

```
logreg_before = LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
logreg_before.fit(X_train_scaled, y_train)
result1 = evaluate_clf(logreg_before, "Logistic Regression (Trước SMOTE)", X_test_scaled, y_test)
```

✓ 0.3s

Hình 3.35: Mã nguồn model logistic regression trước smote

- Khởi tạo mô hình với max_iter=1000 đây là số lần thử tối đa để máy tính tìm ra lời giải toán học tốt nhất. 1000 thường là đủ để đảm bảo mô hình "hội tụ"
- class_weight='balanced' lệnh này báo cho mô hình biết các nhóm có kích thước không bằng nhau. Nó tự động gán "tầm quan trọng" cao hơn cho nhóm thiểu số
- random_state = 42 đảm bảo bạn nhận được cùng một kết quả mỗi khi chạy mã.
- Hàm fit() chính là quá trình "học" thực tế. Mô hình nhìn vào các đặc trưng đã chuẩn hóa (X_train_scaled) và đáp án đúng (y_train) để tìm ra các quy luật dẫn đến bệnh tiểu đường.

==== Logistic Regression (Trước SMOTE) ====

Accuracy : 0.8102

Precision: 0.2170

Recall : 0.7612

F1-score : 0.3377

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.81	0.89	987
1	0.22	0.76	0.34	67
accuracy			0.81	1054
macro avg	0.60	0.79	0.61	1054
weighted avg	0.93	0.81	0.85	1054

Hình 3.36: Kết quả model logistic regression trước smote

- Sau khi smote dữ liệu :

```
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)
print(f"Sau SMOTE:")
logreg_after = LogisticRegression(max_iter=1000, random_state=42)
logreg_after.fit(X_train_smote, y_train_smote)
result2 = evaluate_clf(logreg_after, "Logistic Regression (Sau SMOTE)", X_test_scaled, y_test)
```

✓ 0.4s

Hình 3.37: Mã nguồn model logistic regression sau smote

- Sử dụng `smote.fit_resample(X_train_scaled, y_train)` lên tập dữ liệu huấn luyện đảm bảo mô hình không bị mất cân bằng dữ liệu khi dự đoán

==== Logistic Regression (Sau SMOTE) ====

Accuracy : 0.7884

Precision: 0.2023

Recall : 0.7910

F1-score : 0.3222

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.79	0.87	987
1	0.20	0.79	0.32	67
accuracy			0.79	1054
macro avg	0.59	0.79	0.60	1054
weighted avg	0.93	0.79	0.84	1054

Hình 3.38: Kết quả model logistic regression sau smote

- Đây là kết quả chuẩn xác nhất khi mô hình được học qua các mẫu dữ liệu cân bằng và đưa ra chuẩn đoán

3.4.2 Model Random Forest

```
X_res, y_res = smote.fit_resample(X_train_scaled, y_train)
rf = RandomForestClassifier(n_estimators=800, random_state=42, n_jobs=-1)
rf.fit(X_res, y_res)
metrics = evaluate_clf(rf, 'Random Forest', X_test_scaled, y_test)
model_metrics = pd.concat([model_metrics, pd.DataFrame([metrics])], ignore_index=True)
```

Hình 3.39: Mã nguồn model random forest

- Sử dụng `smote.fit_resample` lên tập dữ liệu trước khi áp dụng model sử dụng random forest từ thư viện `sklearn`.
- Khởi tạo random forest với `n_estimators=800` Mô hình sẽ xây dựng 800 cây quyết định khác nhau và kết hợp câu trả lời của chúng. Càng nhiều cây, kết quả thường càng ổn định.
- `n_jobs=-1` Yêu cầu máy tính sử dụng tất cả các nhân CPU có sẵn để tăng tốc độ huấn luyện.
- Mô hình "học" từ dữ liệu đã cân bằng (`X_res`). Sau đó, hàm `evaluate_clf` kiểm tra hiệu suất của nó trên tập `X_test_scaled` (dữ liệu thực tế mà nó chưa từng thấy).
- Nó lấy các điểm số hiệu suất (Accuracy, Precision, Recall, F1) của mô hình Random Forest và thêm chúng thành một hàng mới vào bảng tổng hợp `model_metrics`.

3.4.3 So sánh và đánh giá mô hình

SO SÁNH 2 MÔ HÌNH SAU SMOTE				
	accuracy	precision	recall	f1
modelname				
Random Forest	0.9137	0.3462	0.4030	0.3724
Logistic	0.7884	0.2023	0.7910	0.3222

Hình 3.40: So sánh hai model

- Mô hình Logistic Regression:
 - Ưu điểm: Có chỉ số Recall cực cao (0.7910). Trong y tế, điều này rất quan trọng vì nó có nghĩa là mô hình tìm được gần 80% số người thực sự bị tiểu đường.
 - Nhược điểm: Precision thấp (0.2023), nghĩa là mô hình này bị "nhảy cảm quá mức", dẫn đến việc dự đoán nhầm rất nhiều người khỏe mạnh là có bệnh.
- Mô hình Random Forest:
 - Ưu điểm: Độ chính xác tổng thể (Accuracy) rất cao. Nó hoạt động ổn định và ít gây ra báo động giả hơn Logistic.
 - Nhược điểm: Recall thấp (0.4030). Mô hình này bỏ sót tới 60% bệnh nhân thực tế. Đây là một điểm yếu chí mạng trong bài toán chẩn đoán bệnh.

Tuy nhiên trong thực tế việc bỏ sót bệnh nhân là một việc vô cùng quan trọng trong lĩnh vực y tế vì nó liên quan trực tiếp đến vấn đề tính mạng con người, trong logistic regression có điểm Recall cao nhất gần 80% trong khi đó random forest chỉ khoảng 40% tức nó bỏ sót tận 60% các trường hợp thật sự bị bệnh rất nguy hiểm vậy theo em ưu tiên sử dụng model logistic regression để thực hiện dự đoán bệnh nhân có nguy cơ mắc bệnh tiểu đường.

3.5 Sản phẩm



Hình 3.41: Tiêu đề sản phẩm

Nhập Thông Tin Sức Khỏe

Tuổi (Age)
Ví dụ: 45

Giới tính
Nam

Chiều cao (m)
Ví dụ: 1.70

Cân nặng (kg)
Ví dụ: 65

BMI (Tỷ trọng)

Đường huyết (mmol/L)
Ví dụ: 5.5

Nhịp tim
Ví dụ: 75

Huyết áp tâm thu
Ví dụ: 120

Huyết áp tâm trương
Ví dụ: 80

Tiền sử tiểu đường gia đình?
Không

Cá nhân mắc tăng huyết áp?
Không

Tiền sử tăng huyết áp gia đình?
Không

Mắc bệnh tim mạch?
Không

Tiền sử đột quỵ?
Không

PHÂN TÍCH KẾT QUẢ

Hình 3.42: Giao diện sản phẩm

- Phần này được sử dụng để dự đoán nguy cơ mắc bệnh tiểu đường của người nhập gồm các thông tin sức khỏe và tiền sử mắc bệnh của gia đình.

Nhập Thông Tin Sức Khỏe

Tuổi (Age)
21

Giới tính
Nam

Chiều cao (m)
1.80

Cân nặng (kg)
66

BMI (Tự động)
20.37

Đường huyết (mmol/L)
5.7

Nhịp tim
75

Huyết áp tâm thu
120

Huyết áp tâm trương
84

Tiền sử tiểu đường gia đình?
Không

Cá nhân mắc tăng huyết áp?
Không

Tiền sử tăng huyết áp gia đình?
Không

Mắc bệnh tim mạch?
Không

Tiền sử đột quỵ?
Không

PHÂN TÍCH KẾT QUẢ

Kết quả dự đoán:
12.40%

☒ AN TOÀN: Các chỉ số hiện tại ở mức ổn định.

Làm lại

Hình 3.43: Kết quả chạy của một ví dụ dự đoán bằng logistic regression

KẾT LUẬN

Bài tập lớn môn Học máy đã được hoàn thành với việc nghiên cứu và trình bày cơ sở lý thuyết vững chắc về quy trình Khai phá dữ liệu (Data Mining), kỹ thuật tiền xử lý (chuẩn hóa, SMOTE) và các thuật toán phân loại nòng cốt (Logistic Regression, Random Forest). Quan trọng hơn, đề tài đã áp dụng thành công các kiến thức này vào việc xây dựng hệ thống hỗ trợ chẩn đoán y tế thực tế:

Xử lý và cân bằng dữ liệu: Đã sử dụng hiệu quả kỹ thuật SMOTE để giải quyết bài toán mất cân bằng dữ liệu, giúp mô hình học được đặc trưng của nhóm bệnh nhân tiểu đường một cách công bằng, khắc phục hạn chế của việc mô hình bị thiên kiến về nhóm khỏe mạnh

Huấn luyện và đánh giá mô hình: Đã cài đặt và so sánh chi tiết hai thuật toán. Kết quả thực nghiệm cho thấy sự đánh đổi rõ rệt: Random Forest đạt độ chính xác tổng thể cao (~91%) phù hợp cho chẩn đoán chắc chắn, trong khi Logistic Regression lại vượt trội về độ nhạy (Recall ~79%), chứng minh khả năng tối ưu cho mục đích tầm soát diện rộng (không bỏ sót bệnh)

Ứng dụng thực tiễn: Xây dựng thành công giao diện Web trực quan, cho phép người dùng nhập 14 chỉ số sức khỏe và nhận cảnh báo nguy cơ tức thì. Qua đó, đề tài không chỉ củng cố kiến thức nền tảng về Học máy mà còn phát triển kỹ năng phân tích dữ liệu, thiết kế hệ thống và tư duy giải quyết bài toán y tế bằng công nghệ.

TÀI LIỆU THAM KHẢO

- 1) Swinburne V.
- 2) Youtube: học lập trình cùng Phát: Machine learning cơ bản Logistic Regression
- 3) Numpy : [Giới thiệu về Numpy - Nguyen Van Hoang \(viblo.asia\)](https://viblo.asia/Giới-thiệu-về-Numpy)
- 4) N.V . Hậu, P. M. Chuẩn, và N. V. Quyết , “*Giáo trình Học máy cơ bản*”, Nhà xuất bản Khoa học và Kỹ thuật, 2022
- 5) GeekforGeeks: random forest algorithm in machine learning